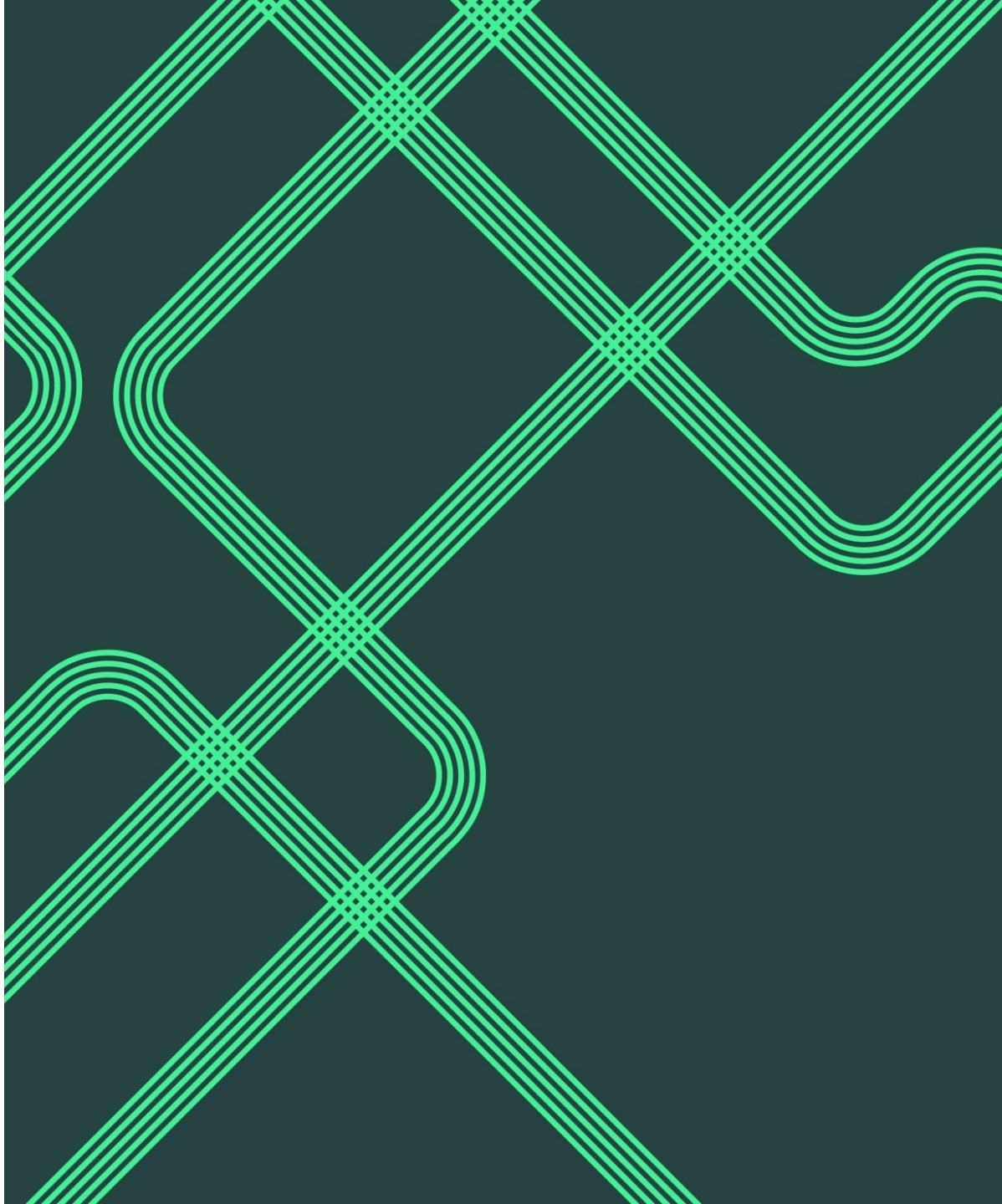
TEMA 67 (II)

Entorno de desarrollo JAVA



ÍNDICE

1. Evolución de la capa web en Java
2. Microservicios y cloud
3. Herramientas de desarrollo
4. Acceso a datos e integración
5. Seguridad en el desarrollo Java
6. Certificaciones
7. Java en la Administración Pública
8. Repaso: ideas clave y preguntas tipo test

1. EVOLUCIÓN DE LA CAPA WEB EN JAVA

Servlets y JSP · Jakarta EE · MVC · Struts · JSF · Spring y Spring Boot · APIs REST

INTRODUCCIÓN Y CONTEXTO

- Punto de partida:
  - Servlets (Java EE): manejo a bajo nivel de peticiones HTTP.
  - JSP (JavaServer Pages): páginas dinámicas con Java embebido.
  - JSF (JavaServer Faces): modelo basado en componentes UI.
- Esquema básico de flujo en Servlets
  - Cliente (navegador) → Petición HTTP → Servlet (lógica) → JSP (vista) → Respuesta HTTP

SERVLETS Y JSP: CONCEPTOS CLAVE

- **Servlet:** clase Java que atiende peticiones HTTP dentro de un contenedor web.
  - Ciclo de vida gestionado por el contenedor: init() una vez, service() en cada petición (doGet, doPost…) y destroy() al retirarlo.
  - Una sola instancia atiende peticiones concurrentes en hilos distintos: no debe guardar estado en atributos.
- **Objetos principales:** HttpServletRequest, HttpServletResponse y HttpSession (estado por usuario).
- **Filtros y listeners:** pre y posprocesado de peticiones (seguridad, trazas) y reacción a eventos del ciclo de vida.
- **Configuración:** descriptor web.xml o anotaciones (@WebServlet, @WebFilter) desde Servlet 3.0.
- **Empaquetado WAR:** WEB-INF/web.xml, WEB-INF/classes y WEB-INF/lib.
- **JSP:** se traduce a un servlet la primera vez que se pide; usa Expression Language (${…}) y JSTL en lugar de código Java embebido.

CONTENEDORES Y SERVIDORES JAVA

Producto	Tipo	Notas
Apache Tomcat	Contenedor web (Servlet, Pages, WebSocket)	El más usado (86%). Tomcat 11 implementa Servlet 6.1 (Jakarta EE 11). Embebido por defecto en Spring Boot.
Eclipse Jetty	Contenedor web ligero	Muy fácil de embeber (16%). Alternativa a Tomcat en Spring Boot.
WildFly / JBoss EAP	Servidor de aplicaciones Jakarta EE	Red Hat (14%). JBoss EAP es la versión con soporte comercial.
Oracle WebLogic	Servidor de aplicaciones comercial	Muy presente en grandes organizaciones (9%).
WebSphere / Open Liberty	Servidor de aplicaciones (IBM)	Liberty es modular e implementa Jakarta EE y MicroProfile (8%).
GlassFish / Payara	Servidor de aplicaciones	GlassFish (Eclipse) fue la implementación de referencia; Payara deriva de él.

Porcentajes de uso: 2025 Java Developer Productivity Report (Perforce/JRebel), respuesta múltiple. Un 14% no usa servidor de aplicaciones.

DE JAVA EE A JAKARTA EE

Versión	Año	Novedades clave
J2EE 1.2	1999	Primera versión: Servlets, JSP, EJB, JMS, JDBC.
Java EE 5	2006	Anotaciones, EJB 3, JPA y JSF 1.2.
Java EE 6	2009	CDI, JAX-RS, Bean Validation y Web Profile.
Java EE 7	2013	WebSocket, JSON-P, Batch y Concurrency Utilities.
Java EE 8	2017	JSON-B, Security API y Servlet 4 (HTTP/2).
Jakarta EE 8	2019	Traspaso de Oracle a la Eclipse Foundation (mismo paquete javax).
Jakarta EE 9	2020	Cambio de espacio de nombres: javax.* → jakarta.*
Jakarta EE 10	2022	Core Profile para microservicios; Java 11 como mínimo.
Jakarta EE 11	2025	Java 17 como mínimo, Jakarta Data, hilos virtuales; sin Security Manager.
Jakarta EE 12	En preparación	Requerirá Java 21 como mínimo; aún no publicada (octubre de 2026).

EL PATRÓN MVC EN JAVA WEB Y LA ERA DE STRUTS

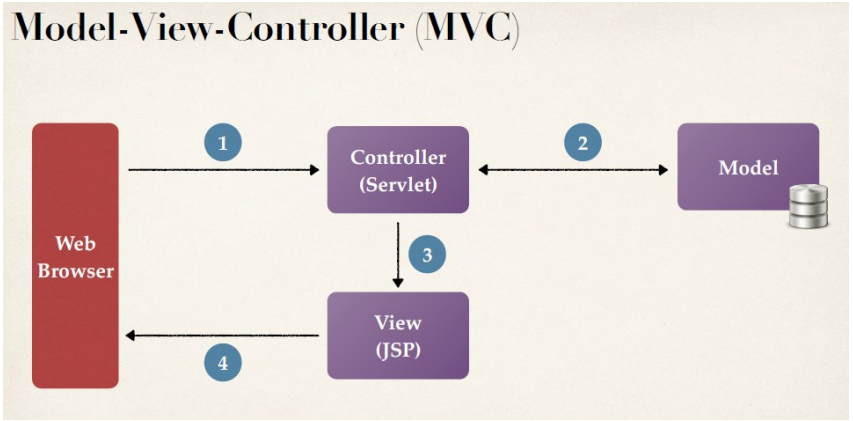
- Contexto inicial (años 2000)
  - Tecnologías base: Servlets + JSP.
  - Problema: mezcla de lógica de negocio y vista: suponía código difícil de mantener.
  - Solución: aplicar patrón MVC en la web.

PATRÓN MVC EN JAVA

- Patrón MVC:
  - Arquitectura que separa la app en Modelo, Vista y Controlador.
  - Beneficios: separación de responsabilidades, bajo acoplamiento, mantenimiento y escalabilidad.
  - Base de frameworks Java clásicos: Struts, JSF, Spring MVC.

MVC CLÁSICO CON SERVLETS Y JSP:

- Modelo: JavaBeans, clases de servicio, DAOs (lógica y datos).
- Vista: JSP (genera HTML dinámico).
- Controlador: Servlets (gestionan la petición y coordinan modelo y vista).
- Flujo típico:
    1. Cliente realiza petición HTTP
    2. Servlet procesa y llama al modelo
    3. Datos se guardan en atributos (request/session)
    4. Servlet reenvía a JSP
    5. JSP genera la respuesta HTML al navegador

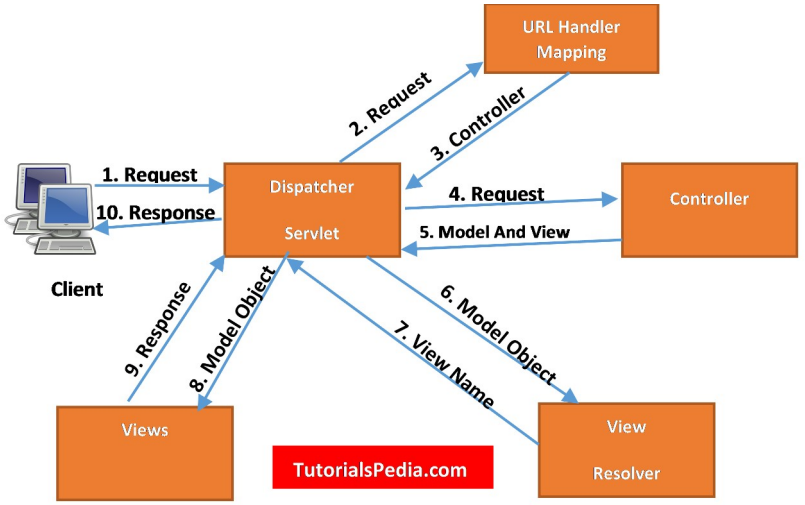


PATRONES DE LA CAPA DE PRESENTACIÓN

Patrón	Qué resuelve	Ejemplos
Front Controller	Un único punto de entrada recibe todas las peticiones y las despacha	DispatcherServlet (Spring), ActionServlet (Struts 1), FacesServlet (JSF)
Intercepting Filter	Pre y posprocesado de peticiones en cadena	Filtros de Servlet, cadena de filtros de Spring Security
View Helper	Sacar la lógica de presentación de la vista	Taglibs, JSTL, beans de apoyo
Composite View	Componer páginas a partir de fragmentos	Plantillas de Facelets, layouts de Thymeleaf, Tiles
DTO	Agrupar datos para transferirlos entre capas	Records de Java como DTO
DAO	Encapsular el acceso a datos	Hoy, repositorios de Spring Data o Jakarta Data
Service Locator	Localizar servicios (JNDI)	Hoy sustituido por la inyección de dependencias

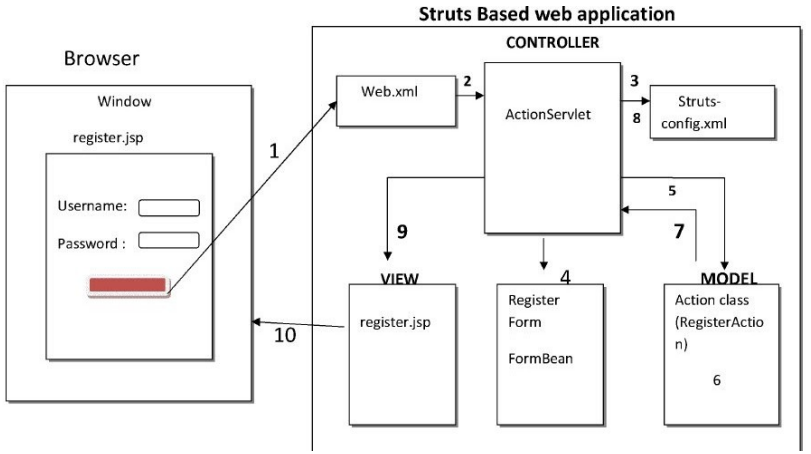
MVC CON SPRING FRAMEWORK

- Implementa el patrón con un Front Controller: DispatcherServlet.
- Anotaciones (@Controller, @RequestMapping, etc.) para definir controladores y rutas.
- Integración con beans de Spring (DI, transacciones, validación).
- Flujo Spring MVC:
    1. El cliente envía la petición al DispatcherServlet
    2. Handler Mapping localiza el controlador
    3. Controlador ejecuta lógica y devuelve datos + nombre de vista
    4. ViewResolver selecciona la plantilla adecuada (JSP, Thymeleaf, etc.)
    5. Vista genera HTML y responde al cliente



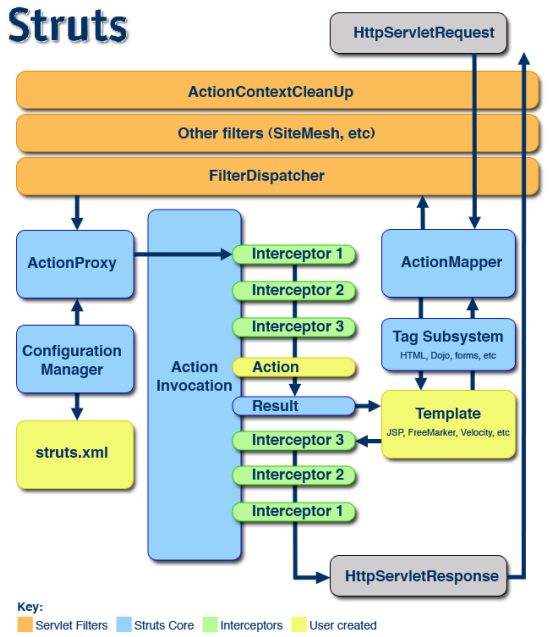
APACHE STRUTS 1

- Introduce un Front Controller (ActionServlet).
- Actions: lógica de control.
- ActionForms: recogen y validan los datos de los formularios.
- JSP + taglibs: vistas.
- Modelo: objetos de negocio.
- Configuración centralizada en struts-config.xml.
- Popular en la era J2EE: usado en multitud de aplicaciones corporativas.



LIMITACIONES Y DECLIVE DE STRUTS

- Formularios y validación rígidos.
- Testing difícil, mucha configuración XML.
- Struts 2 (basado en WebWork) mejoró, pero llegó tarde.
- Struts 1 dejó de mantenerse en 2013.
- Caso crítico: una vulnerabilidad de Struts 2 (CVE-2017-5638) causó la brecha de Equifax en 2017.

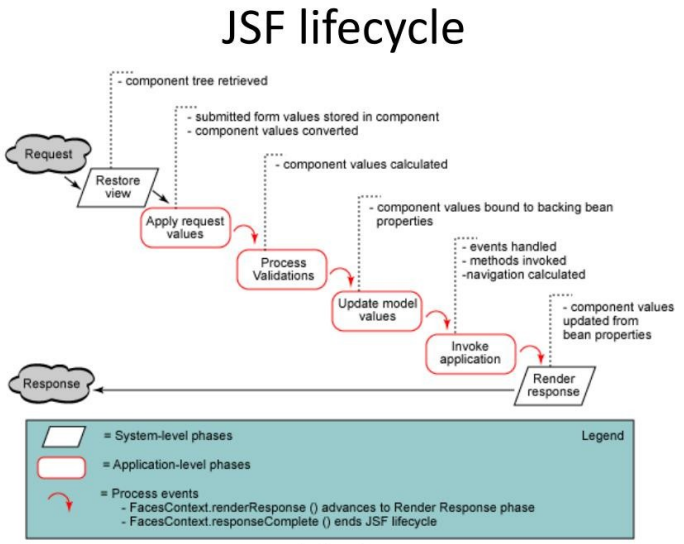


STRUTS 2 Y SITUACIÓN ACTUAL

- Struts 2 nace de WebWork, adoptado por Apache como Struts 2 (primera versión estable en 2007).
  - Actions como POJO, interceptores para la lógica transversal y resultados (JSP, FreeMarker).
  - Usa OGNL para acceder a los datos, origen de muchas vulnerabilidades graves (como la de Equifax, en el analizador multipart).
- Estado de las versiones:
  - Struts 1: fin de vida en 2013.
  - Struts 2.5: fin de vida en abril de 2024.
  - Struts 6 (2022): mantenida.
  - Struts 7 (diciembre de 2024): requiere Java 17 y usa el espacio de nombres jakarta.*
- Recomendación: no usarlo en proyectos nuevos; migrar las aplicaciones heredadas a Spring MVC o, como mínimo, a Struts 7.

JAVASERVER FACES (JSF)

- Propuesta de Sun (JSR 127, 2004): capa de presentación basada en componentes.
- JSF = framework oficial Java EE (estándar); hoy se llama Jakarta Faces.
- Páginas con Facelets (XHTML) + beans gestionados en servidor (hoy beans CDI).
- Manejo integrado de:
  - Conversión y validación de formularios.
  - Navegación entre páginas.
  - Conservación de estado en servidor/cliente.



JSF: CICLO DE VIDA Y ECOSISTEMA

Fase	Qué ocurre
1. Restore View	Reconstruye (o crea) el árbol de componentes de la página
2. Apply Request Values	Cada componente toma su valor de la petición
3. Process Validations	Conversión y validación; si falla, salta a Render Response
4. Update Model Values	Copia los valores válidos a los beans
5. Invoke Application	Ejecuta acciones y navegación
6. Render Response	Genera la respuesta HTML y guarda el estado de la vista

- Implementaciones: Mojarra (Eclipse) y Apache MyFaces. Librerías de componentes: PrimeFaces y OmniFaces.
- Los beans son CDI (@Named con @RequestScoped, @ViewScoped…); @ManagedBean se eliminó en Faces 4.0.

JSF: DECLIVE Y ESTADO ACTUAL

- Aparición de AJAX y luego SPA (Angular, React, Vue).
- JSF genera HTML y controla en servidor, supone un mal encaje con UIs modernas.
- Curva de aprendizaje alta, debugging complejo.
- Más pesado que opciones ligeras (Spring MVC + plantillas).
- Jakarta EE lo mantiene por compatibilidad: Jakarta Faces 4.1 forma parte de Jakarta EE 11.
- Sigue presente en muchas aplicaciones corporativas y de la Administración, a menudo con PrimeFaces.

SPRING MVC

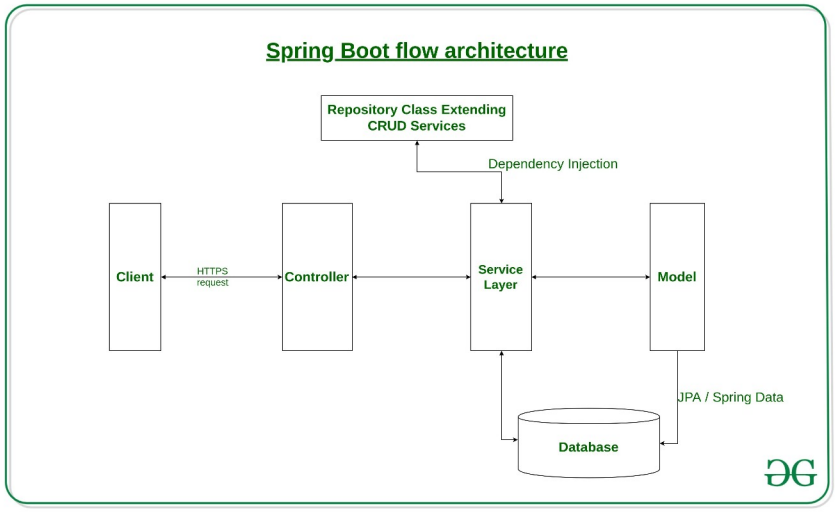
- Nace como parte del Spring Framework (2003).
- MVC con controladores POJO + anotaciones (@Controller, @RequestMapping).
- Integración natural con inyección de dependencias.
- Mucho más ligero y flexible que Struts o JSF.
- Motores de vista: JSP inicialmente y luego Thymeleaf, FreeMarker.

SPRING: INVERSIÓN DE CONTROL Y AOP

- **Contenedor IoC (ApplicationContext):** crea los beans, resuelve sus dependencias y gestiona su ciclo de vida.
- **Inyección de dependencias:** por constructor (la recomendada), por setter o por campo (@Autowired).
- **Definición de beans:** estereotipos @Component, @Service, @Repository y @Controller, o métodos @Bean en clases @Configuration.
- **Ámbitos:** singleton (por defecto) y prototype; en web, request, session y application.
- **AOP:** aspectos para funciones transversales (transacciones, seguridad, trazas): advice (@Before, @After, @Around) y pointcut.
  - Se implementa con proxies: @Transactional no se aplica en llamadas internas de la misma clase.
- **Configuración externa:** perfiles (@Profile), @Value y @ConfigurationProperties.

SPRING BOOT (2014)

- Introduce auto-configuración y starters.
- Arranque con servidor Tomcat embebido.
- Filosofía just run: una clase @SpringBootApplication + controladores permite tener una API lista en minutos.
- Facilita empaquetar apps como fat JARs ejecutables, perfecto para Docker.
- Adopción masiva: hoy es el estándar de facto en web y microservicios Java.



SPRING BOOT: PIEZAS CLAVE

Pieza	Qué aporta
Starters	Dependencias agrupadas por funcionalidad (spring-boot-starter-data-jpa, -security, -test…)
Autoconfiguración	Crea los beans necesarios según las librerías presentes en el classpath; se puede sobrescribir
Configuración externa	application.properties o application.yml, variables de entorno y perfiles (spring.profiles.active)
Servidor embebido	Tomcat por defecto (también Jetty); la aplicación se arranca con java -jar
Actuator	Endpoints de operación: /actuator/health, /metrics, /info, /prometheus
DevTools	Reinicio automático y recarga en caliente durante el desarrollo
Spring Initializr	Generador de proyectos (start.spring.io)
Empaquetado	JAR ejecutable, imagen de contenedor con buildpacks o ejecutable nativo con GraalVM

EJEMPLO: API REST CON SPRING BOOT

```
@RestController
@RequestMapping("/api/empleados")
public class EmpleadoController {
    private final EmpleadoService servicio;

    public EmpleadoController(EmpleadoService servicio) {
        this.servicio = servicio;
    }

    @GetMapping("/{id}")
    public Empleado obtener(@PathVariable Long id) {
        return servicio.buscar(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Empleado crear(@Valid @RequestBody Empleado e) {
        return servicio.guardar(e);
    }
}
```

- Inyección del servicio por constructor.
- @RestController: las respuestas se convierten a JSON.
- @GetMapping y @PostMapping enlazan rutas y métodos HTTP.
- @PathVariable lee la URL; @RequestBody, el cuerpo JSON.
- @Valid aplica Bean Validation (@NotNull, @Size…).
- @ResponseStatus(CREATED) devuelve 201 en lugar de 200.

ECOSISTEMA SPRING

- Spring Security: autenticación/autorización (OAuth2, JWT, LDAP…).
- Spring Data JPA: acceso a datos simplificado con repositorios.
- Spring Cloud: herramientas para microservicios (Config, Eureka, Gateway…).
- Spring WebFlux: programación reactiva y no bloqueante.
- Spring Batch: procesos por lotes (cargas masivas, ficheros).
- Spring Integration: patrones de integración empresarial.
- Spring for GraphQL y Spring gRPC: otros estilos de API.
- Spring Modulith: monolitos modulares bien estructurados.
- Spring AI: integración con modelos de IA generativa (versión 1.0 en 2025).

SPRING HOY (2026)

- Spring Boot 4.0 (noviembre de 2025), sobre Spring Framework 7: Java 17 como mínimo, soporte de Java 25 y base Jakarta EE 11.
  - Spring Boot 4.1 (junio de 2026) es la rama más reciente.
  - Spring Boot 3.5 terminó su soporte gratuito en junio de 2026.
- Uso mayoritario: Spring es el framework que más usan los desarrolladores Java (65% según JetBrains, 2025).
- Administración pública:
  - Licitaciones TIC piden Spring Boot.
  - Proyectos como LexNET renovado (Justicia) o Hacienda lo emplean.
- Competidores: Quarkus, Micronaut y Helidon, con menor adopción.
- Conclusión: Spring Boot es el framework dominante en Java web.

NOVEDADES DE SPRING 7 Y SPRING BOOT 4

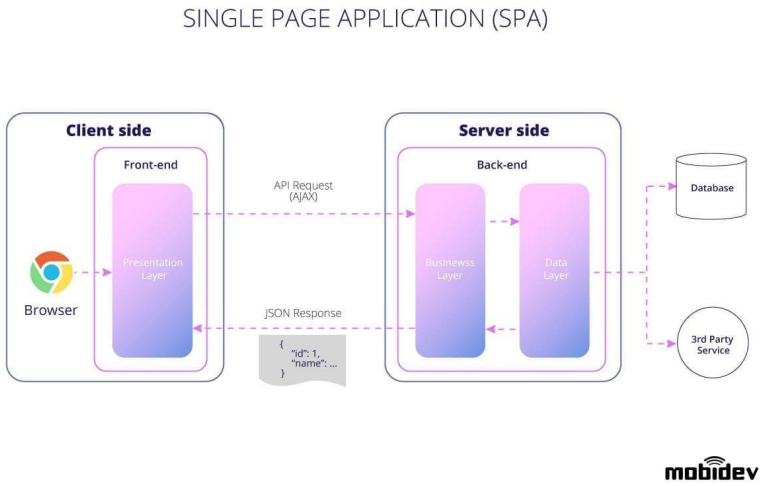
- Versionado de APIs REST integrado (por ruta, cabecera, parámetro o tipo de medio).
- Seguridad frente a nulos con anotaciones JSpecify (@Nullable, @NonNull).
- Resiliencia en el núcleo: @Retryable y @ConcurrencyLimit, antes en Spring Retry.
- Clientes HTTP declarativos (interfaces HTTP) sobre RestClient; RestTemplate va camino de la deprecación.
- Jackson 3 para JSON y autoconfiguración modular, con starters más pequeños (spring-boot-starter-webmvc).
- Se retiran las anotaciones javax.*, el servidor Undertow (no es compatible con Servlet 6.1) y lo marcado como obsoleto en la rama 3.x.
- Spring Authorization Server pasa a formar parte de Spring Security 7.

OTRAS ALTERNATIVAS EN LA CAPA WEB JAVA

- Frameworks component/server-side
  - Vaadin: UI en Java, genera frontend automáticamente (Web Components). Ideal en intranets/dashboards.
  - Apache Wicket: enfoque component-based ligero, separación HTML/Java, nicho reducido.
  - GWT: compilaba Java a JavaScript. Hoy en retroceso.
- Frameworks inspirados en otros lenguajes
  - Play Framework: estilo Rails, APIs en Java/Scala, reactivo. Popularidad caída tras 2015; Play 3 sustituyó Akka por Apache Pekko tras el cambio de licencia de Akka.

TENDENCIA DOMINANTE: FRONTEND JS + BACKEND JAVA

- Angular, React, Vue para aplicaciones SPA modernas.
- Backend Java provee APIs REST/GraphQL.
- Separación de equipos: front (JavaScript/TypeScript, con herramientas de Node.js) y back (Java).



APIS Y COMUNICACIÓN MODERNA

- REST (dominante): @RestController en Spring, Jakarta REST (antes JAX-RS).
- GraphQL: consultas flexibles definidas por el cliente (Spring for GraphQL).
- gRPC: llamadas remotas binarias (Protocol Buffers sobre HTTP/2), habitual entre microservicios; Spring Boot 4.1 incorpora Spring gRPC.
- WebSockets: comunicación bidireccional en tiempo real (paneles, chats).
- SSE (Server-Sent Events): envío unidireccional del servidor al cliente (notificaciones, progreso).
- Complementan a la arquitectura SPA + Java backend.

DISEÑO DE APIS REST: MÉTODOS HTTP

Método	Uso	Seguro	Idempotente
GET	Obtener un recurso o una colección	Sí	Sí
POST	Crear un recurso o lanzar una operación	No	No
PUT	Reemplazar un recurso completo	No	Sí
PATCH	Modificar parcialmente un recurso	No	No (en general)
DELETE	Eliminar un recurso	No	Sí
HEAD / OPTIONS	Solo cabeceras / métodos permitidos (CORS)	Sí	Sí

- **Seguro:** no modifica el estado del servidor. **Idempotente:** repetir la petición produce el mismo efecto que hacerla una vez.
- Modelo de madurez de Richardson: nivel 0 (un único endpoint), 1 (recursos), 2 (verbos HTTP) y 3 (hipermedia, HATEOAS).

CÓDIGOS DE ESTADO Y BUENAS PRÁCTICAS

Código	Significado
200 OK	Petición correcta, con respuesta
201 Created	Recurso creado (cabecera Location)
204 No Content	Correcto, sin cuerpo
400 Bad Request	Petición mal formada o inválida
401 Unauthorized	Falta autenticación o no es válida
403 Forbidden	Autenticado, pero sin permiso
404 Not Found	El recurso no existe
409 Conflict	Conflicto con el estado actual
500 Internal Server Error	Error no controlado en el servidor

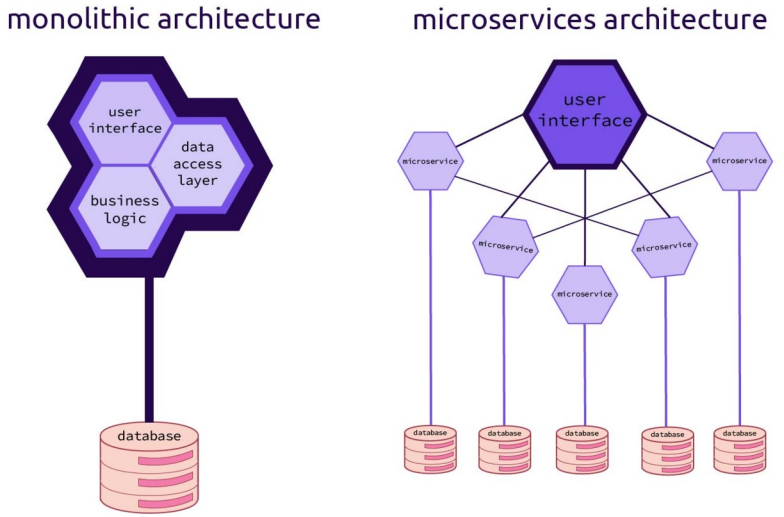
- Recursos con sustantivos en plural: /empleados/42
- Versionado de la API (/v1/… o cabecera)
- Paginación, filtros y orden con parámetros
- Documentación con OpenAPI (springdoc-openapi)
- Errores con formato estándar: Problem Details (RFC 9457), soportado por Spring

2. MICROSERVICIOS Y CLOUD

Microservicios · Spring Cloud · Kubernetes · Observabilidad · Quarkus, Micronaut y MicroProfile

ARQUITECTURA MODERNA: MICROSERVICIOS Y CLOUD EN JAVA

- ¿Qué son los microservicios?
  - Estilo arquitectónico: en lugar de una aplicación, muchos servicios pequeños, autónomos, desplegables.
  - Cada servicio enfocado en una función de negocio.
  - Contraste con monolitos (todos los módulos en un único despliegue).
- Ventajas clave
  - Escalabilidad independiente.
  - Resiliencia frente a fallos.
  - Despliegue continuo y ágil.
  - Cierta flexibilidad tecnológica.



¿QUÉ NO ES UN MICROSERVICIO?

- No es una “app pequeña”, sino un servicio autónomo y enfocado en un dominio de negocio.
- Expone una interfaz clara y mínima (normalmente unos pocos endpoints REST/gRPC o eventos).
- Encapsula su propia lógica y datos, sin compartir base de datos directa con otros servicios.
- La comunicación con el resto del sistema se hace solo a través de su API.
- Ejemplo:
  - Servicio de Pagos: /pagar, /reembolsar, /estado
  - Servicio de Usuarios: /crearUsuario, /consultarPerfil

JAVA Y LA ERA DE LOS MICROSERVICIOS

- Java fue el lenguaje de los grandes monolitos enterprise (WebLogic, EARs).
- Se adaptó rápido al nuevo modelo:
  - JARs independientes en contenedores Docker.
  - Orquestación con Kubernetes, que asume funciones que antes se resolvían con librerías Netflix OSS.
- Razones del encaje:
  - Ecosistema maduro (REST, seguridad, drivers DB).
  - Rendimiento consolidado.
  - Gran comunidad de desarrolladores.

PATRONES DE MICROSERVICIOS

Patrón	Problema que resuelve	En Java
API Gateway	Punto de entrada único: enrutado, seguridad, límites de uso	Spring Cloud Gateway
Service Discovery	Localizar instancias que cambian dinámicamente	Eureka, Consul, DNS de Kubernetes
Circuit Breaker	Evitar fallos en cascada cuando un servicio no responde	Resilience4j, SmallRye Fault Tolerance
Database per Service	Independencia de datos entre servicios	Un esquema o BD por servicio
Saga	Transacciones distribuidas con pasos locales y compensaciones	Orquestación o coreografía con eventos
CQRS	Separar el modelo de escritura del de lectura	Axon Framework
Event Sourcing	Guardar el estado como una secuencia de eventos	Axon, Kafka
Strangler Fig	Migrar un monolito de forma gradual	Gateway que desvía rutas al nuevo servicio
Sidecar	Funciones transversales en un contenedor auxiliar	Service mesh: Istio, Linkerd

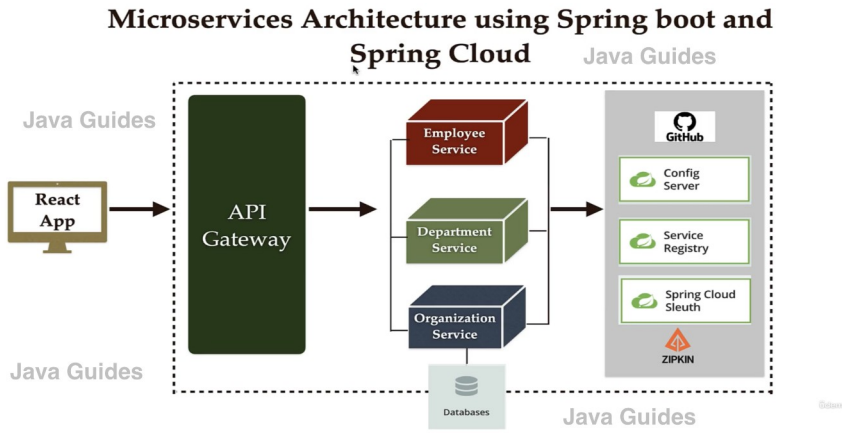
THE TWELVE-FACTOR APP

Factor	Idea	Factor	Idea
I. Código base	Un repositorio por aplicación, muchos despliegues	VII. Puertos	El servicio se expone en su propio puerto (servidor embebido)
II. Dependencias	Declaradas y aisladas (pom.xml, build.gradle)	VIII. Concurrencia	Escalar añadiendo procesos
III. Configuración	En el entorno, nunca en el código	IX. Desechabilidad	Arranque rápido y parada ordenada
IV. Servicios de respaldo	BD, colas o cachés como recursos conectables	X. Paridad dev/prod	Entornos lo más parecidos posible
V. Construir, liberar, ejecutar	Etapas estrictamente separadas	XI. Logs	Flujo de eventos por la salida estándar
VI. Procesos	Sin estado; el estado va a servicios de respaldo	XII. Administración	Tareas puntuales en el mismo entorno

Metodología publicada por Heroku (2011) para aplicaciones cloud-native; Spring Boot y Kubernetes facilitan cumplirla.

RETOS Y SOLUCIONES EN MICROSERVICIOS JAVA

- Arranque más rápido: GraalVM Native Image, Project Leyden, frameworks ligeros.
- Menor consumo de recursos: Quarkus, Micronaut optimizan memoria y tiempos de arranque.
- Observabilidad: métricas con Micrometer y trazas distribuidas con OpenTelemetry o Micrometer Tracing (Zipkin, Jaeger).
- Mensajería distribuida: integración con Kafka, RabbitMQ, ActiveMQ.
- Poliglotía en BBDD: drivers y soporte Spring Data (Mongo, Cassandra, Redis…).
- Concurrencia: hilos virtuales (Java 21) para atender muchas peticiones bloqueantes.



HILOS VIRTUALES (JAVA 21)

- Incorporados de forma definitiva en Java 21 (JEP 444, Project Loom).
- Son hilos ligeros gestionados por la JVM: se pueden crear millones y, al bloquearse en E/S, liberan el hilo del sistema operativo.
- Permiten el estilo imperativo de "un hilo por petición" con una escalabilidad cercana a la programación reactiva.
- Uso:
  - Executors.newVirtualThreadPerTaskExecutor()
  - Spring Boot: spring.threads.virtual.enabled=true; Quarkus: @RunOnVirtualThread; Helidon 4 se basa en ellos.
- Java 24 eliminó el bloqueo del hilo del sistema dentro de synchronized (JEP 491).
- No aportan en tareas intensivas en CPU; la programación reactiva (WebFlux) sigue siendo útil para flujos con contrapresión.
- Relacionado: Scoped Values (definitivos en Java 25) y concurrencia estructurada (aún en preview).

ARRANQUE RÁPIDO: GRAALVM Y LEYDEN

- Problema: la JVM tarda en arrancar y en "calentarse" (JIT), algo que penaliza el autoescalado y el serverless.
- **GraalVM Native Image:** compila la aplicación por adelantado (AOT) a un ejecutable nativo.
  - Arranque en milisegundos y menos memoria.
  - A cambio: compilación lenta y "mundo cerrado" (la reflexión hay que declararla).
  - Lo soportan Quarkus, Micronaut, Helidon y Spring Boot (desde la versión 3).
- En septiembre de 2025 Oracle desligó GraalVM de sus productos Java SE y centró la mejora del arranque en Project Leyden.
- **Project Leyden (OpenJDK):** caché AOT con clases ya cargadas y perfiles de una ejecución de entrenamiento (Java 24 y 25), sin salir de la JVM estándar.
- **CRaC:** restaura una JVM ya arrancada desde una instantánea; lo soportan Spring y algunas distribuciones del JDK.

SPRING CLOUD NETFLIX (2015)

- Librerías:
  - Eureka: descubrimiento de servicios.
  - Ribbon: balanceo cliente.
  - Hystrix: circuit breaker.
  - Zuul: API Gateway.
- Se usaban junto a Spring Cloud Config (que no es de Netflix) para la configuración centralizada.
- Gran impacto: permitió montar entornos de microservicios enteramente en Java.
- Evolución: Ribbon, Hystrix y Zuul se retiraron en Spring Cloud 2020.0 y se sustituyen por Spring Cloud LoadBalancer, Resilience4j y Spring Cloud Gateway. Eureka sigue mantenido.

SPRING CLOUD HOY

- Con Kubernetes, parte de las funciones ya las cubre la plataforma:
  - Descubrimiento: Services y DNS de Kubernetes.
  - Configuración: ConfigMaps y Secrets.
  - Resiliencia básica: sondas de salud y reinicio de pods.
- Pero Spring Cloud sigue aportando:
  - Config Server (centralización en Git).
  - Gateway (versiones sobre WebFlux y sobre Spring MVC).
  - Circuit Breaker (Resilience4j) y Stream (Kafka, RabbitMQ).
- Del resto del ecosistema Spring:
  - Actuator + Micrometer (Spring Boot): métricas y health checks (Prometheus).
  - Spring Authorization Server (OAuth2, OIDC), integrado en Spring Security 7.

KUBERNETES PARA DESARROLLADORES

Objeto	Para qué sirve
Pod	Unidad mínima de ejecución: uno o varios contenedores
Deployment	Mantiene N réplicas de un pod y gestiona las actualizaciones progresivas
Service	Nombre y dirección estables para un conjunto de pods (balanceo interno)
Ingress / Gateway API	Expone servicios HTTP al exterior; Gateway API es su evolución
ConfigMap / Secret	Configuración y datos sensibles inyectados como variables o ficheros
HorizontalPodAutoscaler	Ajusta el número de réplicas según CPU, memoria u otras métricas
Sondas (liveness, readiness)	Indican si el contenedor está vivo o listo; en Spring Boot, /actuator/health/liveness y /readiness
Helm	Gestor de paquetes (charts) para desplegar aplicaciones

OBSERVABILIDAD

Pilar	Qué responde	Herramientas en Java
Logs	¿Qué ha pasado?	SLF4J con Logback o Log4j 2; logs estructurados en JSON; MDC con el identificador de traza; ELK o Loki
Métricas	¿Cómo se comporta el sistema?	Micrometer + Prometheus y Grafana; JMX
Trazas	¿Por dónde pasa una petición?	OpenTelemetry o Micrometer Tracing; Jaeger, Zipkin o Grafana Tempo
Perfilado	¿Dónde se consume CPU y memoria?	JDK Flight Recorder (JFR), JDK Mission Control, async-profiler

- OpenTelemetry (CNCF) es el estándar abierto que unifica trazas, métricas y logs; sustituye a OpenTracing y OpenCensus, ya archivados.
- Spring Boot 3 sustituyó Spring Cloud Sleuth por Micrometer Tracing.

DEVOPS Y DESPLIEGUE EN CLOUD

- Pipelines CI/CD (Jenkins, GitLab CI, GitHub Actions…).
- Creación de imágenes de contenedor:
  - Jib: imágenes sin Dockerfile ni demonio Docker.
  - Eclipse JKube (sucesor del Fabric8 Maven Plugin).
  - Cloud Native Buildpacks (Paketo).
  - Spring Boot soporta buildpacks nativamente: mvn spring-boot:build-image
- Integración con Kubernetes gestionados (EKS, AKS, GKE, OpenShift) y plataformas enterprise (Cloud Foundry, Tanzu).

QUARKUS

- Open source, creado por Red Hat (2019); en 2024 anunció su paso a la Commonhaus Foundation.
- Diseñado para containers y serverless.
- Claves:
  - Mucho trabajo en compile-time: menos reflexión en runtime.
  - Integración estrecha con GraalVM (native image).
  - Núcleo Vert.x (IO reactivo), pero admite estilo imperativo.
  - Implementa APIs conocidas (Jakarta Persistence, Jakarta REST, MicroProfile…).
- Rendimiento: arranque en <100 ms, memoria <30 MB en nativo.
- Soporte comercial de Red Hat (Red Hat build of Quarkus), integrado en OpenShift.

COMPARATIVA SPRING BOOT VS QUARKUS

Característica	Spring Boot	Quarkus
Origen y respaldo	Spring (2014); VMware, hoy Broadcom	Red Hat (2019); Commonhaus Foundation
Inyección de dependencias	En ejecución (reflexión y proxies)	En compilación (ArC, basado en CDI)
Estándares	APIs propias de Spring	Jakarta EE y MicroProfile
Tiempo de arranque*	2–3 segundos (JVM)	< 0,1 segundos (nativo con GraalVM)
Memoria usada (RAM)*	100–150 MB (JVM)	20–30 MB (nativo)
Formato de despliegue	JAR ejecutable o binario nativo	JAR (JVM) o binario nativo
Elasticidad en Kubernetes	Buena, pero más pesada en JVM	Muy alta, ideal para escalar
Productividad	Excelente (ecosistema amplio)	Alta (modo dev con recarga rápida)

** Cifras orientativas que comparan Spring Boot en JVM con Quarkus nativo. Spring Boot también puede compilarse a nativo y Quarkus ejecutarse en JVM: hay que comparar en el mismo modo.*

MICRONAUT: AOT Y MODULARIDAD

- Nacido en 2018 (Object Computing, creadores de Grails).
- Enfoque: IoC en compile-time con procesadores de anotaciones.
- Características:
  - Arranques ~1s en JVM, aún menos en nativo.
  - Casi cero reflexión es ideal para GraalVM.
  - Soporta Java, Kotlin, Groovy.
  - Ecosistema: Micronaut Data, HTTP Client, módulos Kafka, Rabbit, gRPC.
  - Orientado a funciones serverless (AWS Lambda, Azure Functions, Google Cloud Functions).

MICROPROFILE Y OTROS RUNTIMES

- MicroProfile = especificaciones abiertas para microservicios (Eclipse Foundation): Config, Fault Tolerance, Health, JWT, OpenAPI, REST Client y Telemetry.
  - Versión 7.1 (junio de 2025), sobre el Core Profile de Jakarta EE.
- Lo implementan Quarkus, Helidon, Open Liberty, Payara y WildFly.
- Ventaja: menor lock-in, permite portabilidad entre runtimes.
- Oracle impulsa Helidon (microservicios Java cloud-native); Helidon 4 se basa en hilos virtuales.
- En la AGE: proyectos nuevos empiezan a valorar Quarkus/MicroProfile para ahorro de recursos, supone una reducción de costes en K8s.

FRAMEWORKS CLOUD-NATIVE

Aspecto	Spring Boot	Quarkus	Micronaut	Helidon
Respaldo	Broadcom (VMware)	Red Hat; Commonhaus	Micronaut Foundation	Oracle
Inyección de dependencias	En ejecución	En compilación	En compilación	CDI (versión MP)
Estándares	Propios de Spring	Jakarta EE y MicroProfile	Propios	MicroProfile (MP) o API propia (SE)
Ejecutable nativo	Sí (desde 3.0)	Sí, muy optimizado	Sí	Sí
Punto fuerte	Ecosistema y adopción	Arranque, memoria y Kubernetes	Serverless y bajo consumo	Hilos virtuales y respaldo de Oracle

PATRONES Y HERRAMIENTAS EN MICROSERVICIOS JAVA

- Circuit Breaker y resiliencia
  - Resilience4j (@CircuitBreaker, retries, bulkheads).
  - Quarkus: SmallRye Fault Tolerance (MicroProfile FT).
- Tracing distribuido
  - OpenTelemetry (OpenTracing está archivado).
  - Integración con Jaeger/Zipkin: cada request lleva su trace-id.
- Configuración centralizada
  - Spring Cloud Config (refresco dinámico vía /actuator/refresh).
  - MicroProfile Config API (Quarkus, Helidon).
- Service Mesh
  - Istio, Linkerd en Kubernetes: seguridad mTLS, retries, observabilidad.
  - Descargan parte de la resiliencia al sidecar, la lógica sigue en la app.

TESTING, MONITORING Y USO EN LA AGE

- Testing de microservicios
  - Testcontainers permite levantar dependencias reales en Docker.
  - WireMock permite simular servicios REST.
  - Pact JVM y Spring Cloud Contract permiten contract testing.
- Monitorización y rendimiento
  - JDK Flight Recorder (JFR).
  - APMs: Dynatrace, AppDynamics (nivel transacción); Prometheus y Grafana como opción abierta.
- Casos en la Administración
  - Notifica: rediseñado con microservicios.
  - Cl@ve: identificación, firma, autenticación concertada como servicios separados.

3. HERRAMIENTAS DE DESARROLLO

JDK · IDE · Maven y Gradle · Pruebas · CI/CD · Calidad de código · Git

VERSIONES DE JAVA (LTS)

Versión	Año	Novedades destacadas
Java 8	2014	Lambdas, streams y la API de fechas java.time
Java 11	2018	Cliente HTTP estándar, ejecución directa de ficheros .java; salen del JDK Java EE y CORBA
Java 17	2021	Records, clases selladas, bloques de texto, pattern matching en instanceof
Java 21	2023	Hilos virtuales, pattern matching en switch, record patterns, colecciones secuenciadas
Java 25	2025	Scoped Values, ficheros fuente compactos, cabeceras de objeto compactas, mejoras AOT (Leyden)

- Una versión cada 6 meses (marzo y septiembre) y una LTS cada 2 años. La última es Java 27 (septiembre de 2026); la próxima LTS prevista es Java 29 (2027).
- Uso según JetBrains (2025): Java 21 (40%), 17 (39%), 8 (31%) y 11 (17%).

DISTRIBUCIONES Y LICENCIAS DEL JDK

- OpenJDK es la implementación de referencia, de código abierto (GPL con Classpath Exception).
- Distribuciones gratuitas basadas en OpenJDK:
  - Eclipse Temurin (Adoptium), Amazon Corretto, Azul Zulu, Microsoft Build of OpenJDK, Red Hat build of OpenJDK, BellSoft Liberica.
  - Las compilaciones de Oracle en jdk.java.net solo reciben actualizaciones durante 6 meses.
- Oracle JDK:
  - Gratuito con la licencia NFTC durante un periodo limitado tras cada LTS.
  - Después, las actualizaciones requieren suscripción (Java SE Universal Subscription, por empleado).
- JRE: Oracle dejó de publicarlo por separado en Java 11 (algunas distribuciones, como Temurin, aún lo ofrecen); jlink crea entornos de ejecución a medida.
- En contenedores conviene usar imágenes base pequeñas con el JDK o un runtime de jlink.

HERRAMIENTAS INCLUIDAS EN EL JDK

Herramienta	Función
javac	Compila el código fuente (.java) a bytecode (.class)
java	Lanza la JVM; desde Java 11 ejecuta directamente un fichero .java
jar	Empaqueta clases y recursos en ficheros JAR
javadoc	Genera documentación HTML a partir de los comentarios
jshell	Consola interactiva (REPL), desde Java 9
jdeps	Analiza las dependencias entre clases y módulos
jlink	Crea una imagen de ejecución con solo los módulos necesarios
jpackage	Genera instaladores nativos (.msi, .deb, .dmg), desde Java 16
jcmd, jstack, jmap	Diagnóstico de una JVM en ejecución (hilos, memoria)
jfr	Grabaciones de JDK Flight Recorder (se analizan con JDK Mission Control)
keytool, jarsigner	Gestión de claves y certificados; firma de ficheros JAR

HERRAMIENTAS DE DESARROLLO EN JAVA

- IDEs más utilizados (2026)
  - El IDE es clave en la productividad del desarrollador.
  - Principales opciones (informe de Perforce/JRebel 2025, respuesta múltiple):
    - IntelliJ IDEA (84%): líder en la industria.
    - VS Code (31%): editor ligero en auge.
    - Eclipse IDE (28%): aún presente en corporaciones/gobierno.
    - Apache NetBeans (uso residual, sobre todo en educación/legado).
  - Tendencia: asistentes de IA integrados en el IDE (GitHub Copilot, JetBrains AI…).

INTELLIJ IDEA (JETBRAINS)

- 84% de los desarrolladores Java lo usan (Perforce/JRebel, 2025); 72% en la encuesta de JetBrains (2025).
- Desde la versión 2025.3 hay una única distribución: las funciones de la antigua Community son gratuitas y las avanzadas requieren suscripción Ultimate (estándar en empresas).
- Fortalezas:
  - Refactorizaciones seguras, autocompletado avanzado.
  - Integración con Spring, JPA, servidores.
  - Plugins para Docker, Kubernetes, SonarQube for IDE.
  - Debugger potente, profiling, asistente de IA.

ECLIPSE IDE

- Antiguo estándar (dominó 2000–2010).
- Aún ~28% de uso, sobre todo en entornos corporativos.
- Ventajas:
  - Gratuito, extensible, soporte enterprise; nueva versión cada trimestre.
  - Ecosistema de plugins: m2e (Maven), Buildship (Gradle), Spring Tools, JBoss Tools.
- Inconvenientes:
  - Interfaz más compleja, puede volverse pesado.
  - Menor tracción frente a IntelliJ.
- La Eclipse Foundation mantiene también Theia y Che, entornos de desarrollo en navegador y en la nube.

VISUAL STUDIO CODE

- Editor ligero convertido en entorno Java usable.
- Extensiones: Language Support for Java (Red Hat), Extension Pack for Java (Microsoft).
- Fortalezas:
  - Rápido, multiplataforma, bajo consumo.
  - Ideal para proyectos pequeños, polyglot, microservicios.
  - Integración creciente con Spring Boot, Quarkus.
- Uso: 31% (Perforce/JRebel, 2025) y subiendo.
- Limitaciones: refactorización menos profunda que IntelliJ.

APACHE NETBEANS

- Proyecto Apache (antes Sun/Oracle).
- Ventajas:
  - GUI Builder para Swing, RCP.
  - Gratis y sencillo de usar.
- Desventajas:
  - Comunidad pequeña, menos ágil en adoptar tendencias.
  - Uso residual (<5%), más en educación y proyectos antiguos.

HERRAMIENTAS DE CONSTRUCCIÓN EN JAVA

- Objetivo: gestionar dependencias, compilación, empaquetado.
- Antes: javac manual, Ant scripts.
- Hoy: Maven y Gradle dominan el ecosistema.
- Ambos soportados por todos los IDEs y CI/CD modernos.

APACHE MAVEN (2004)

- Modelo declarativo mediante el uso del pom.xml.
- Convenciones estándar de directorios (src/main/java).
- Introdujo Maven Central (repositorio global).
- Fortalezas:
  - Estabilidad, previsibilidad.
  - Estándar de facto en empresas/AGE.
  - Compatible con casi todas las herramientas.
- Plugins clave: maven-compiler-plugin, surefire, failsafe, shade/assembly, cargo.
- Versiones estables: 3.9 y 3.10. Maven 4 sigue en fase de versiones candidatas: exigirá Java 17 y separa el POM de construcción del que se publica.

MAVEN: CICLO DE VIDA Y ÁMBITOS

Fase	Qué hace
validate	Comprueba que el proyecto es correcto
compile	Compila el código fuente
test	Pruebas unitarias (Surefire)
package	Empaqueta en JAR o WAR
verify	Pruebas de integración (Failsafe) y comprobaciones
install	Copia el artefacto al repositorio local (~/.m2)
deploy	Lo publica en el repositorio remoto

Ámbito	Uso
compile	Por defecto: compilación y ejecución
provided	Lo aporta el servidor (p. ej., API de Servlet)
runtime	Solo en ejecución (p. ej., driver JDBC)
test	Solo para compilar y ejecutar pruebas
system	JAR local con ruta fija (desaconsejado)
import	Importa un BOM en dependencyManagement

- Ciclos: clean, default y site. Ejecutar una fase ejecuta todas las anteriores: mvn package compila y prueba antes de empaquetar.

GRADLE (2009)

- DSL basado en Groovy o Kotlin (build.gradle / build.gradle.kts); Kotlin DSL por defecto en los proyectos nuevos.
- Scripts concisos, programables.
- Ventajas:
  - Compilación incremental, build cache y configuration cache.
  - Multi-módulos más fáciles (config programática).
  - Oficial en Android Studio.
- Gradle 9 (julio de 2025): requiere Java 17 para ejecutarse.
- Plugins clave: java, application, spring-boot, war, spotbugs.
- Fortalezas en proyectos grandes y modernos (ej. Spring Framework, Spring Boot e Hibernate usan Gradle).

COMPARATIVA MAVEN VS GRADLE

- Sintaxis: Maven XML rígido / Gradle DSL flexible.
- Velocidad: Gradle (incremental/daemon) suele ganar; Maven ha mejorado.
- Popularidad: Maven sigue liderando (67% según JetBrains, 2025); Gradle es la segunda opción y domina en Android.
- Gestión de versiones: Maven resuelve conflictos con la dependencia más cercana; Gradle, con la versión más alta.
- Integración: ambos soportados en IDEs y pipelines CI/CD.
- AGE: tradición fuerte con Maven; Gradle en crecimiento (menos recelo con Kotlin DSL).

DEPENDENCIAS EN MAVEN Y GRADLE

Maven (pom.xml)

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
  <scope>runtime</scope>
</dependency>
```

Gradle – Groovy DSL (build.gradle)

```
dependencies {
  runtimeOnly 'com.mysql:mysql-connector-j:8.4.0'
}
```

Gradle – Kotlin DSL (build.gradle.kts)

```
dependencies {
  runtimeOnly("com.mysql:mysql-connector-j:8.4.0")
}
```

- Desde la versión 8.0.31 el conector de MySQL se publica como com.mysql:mysql-connector-j; el antiguo mysql:mysql-connector-java ya no se actualiza.
- El ámbito runtime de Maven equivale a runtimeOnly en Gradle; para compilar contra la librería se usa implementation.

DEPENDENCIAS Y REPOSITORIOS

- **Coordenadas GAV:** groupId:artifactId:version identifican cada artefacto.
- **Dependencias transitivas:** se resuelven solas; mvn dependency:tree y gradle dependencies muestran el árbol completo.
- **BOM (Bill of Materials):** fija versiones coherentes para un conjunto de librerías (p. ej., spring-boot-dependencies).
- **Versiones:** SNAPSHOT (en desarrollo) frente a release (inmutables); versionado semántico MAYOR.MENOR.PARCHE.
- **Repositorios:** local (~/.m2), Maven Central y repositorios corporativos (Nexus, Artifactory) que hacen de proxy y guardan los artefactos propios.
- **Wrapper (mvnw, gradlew):** fija la versión de la herramienta en el propio proyecto.

PRUEBAS EN JAVA

- JUnit: estándar en pruebas unitarias.
  - JUnit 6 (septiembre de 2025) requiere Java 17 y unifica la numeración; mantiene el modelo de JUnit 5 (Jupiter), aún muy extendido.
  - Anotaciones @Test, parametrización, extensiones.
  - Integración con Spring Boot Test.
- Librerías complementarias:
  - Mockito: mocks de dependencias.
  - AssertJ/Hamcrest: aserciones legibles.
  - Arquillian, Spring Test: pruebas de integración.
  - Testcontainers: dependencias reales en Docker.

JUNIT: ANOTACIONES PRINCIPALES

JUnit 5 / 6	JUnit 4	Uso
@Test	@Test	Marca un método de prueba
@BeforeEach / @AfterEach	@Before / @After	Antes / después de cada prueba
@BeforeAll / @AfterAll	@BeforeClass / @AfterClass	Una vez por clase (métodos static)
@Disabled	@Ignore	Desactiva una prueba
@DisplayName	—	Nombre descriptivo en los informes
@ParameterizedTest + @ValueSource, @CsvSource…	@RunWith(Parameterized.class)	Misma prueba con varios conjuntos de datos
@Nested	—	Agrupa pruebas en clases internas
@Tag	@Category	Clasifica y filtra pruebas
@ExtendWith	@RunWith	Registra extensiones (MockitoExtension, SpringExtension)

EJEMPLO: PRUEBA CON JUNIT Y MOCKITO

```
@ExtendWith(MockitoExtension.class)
class EmpleadoServiceTest {

    @Mock EmpleadoRepository repositorio;
    @InjectMocks EmpleadoService servicio;

    @Test
    @DisplayName("Sube el salario un 10 %")
    void subeSalario() {
        var ana = new Empleado(1L, "Ana",
                               new BigDecimal("3000"));
        when(repositorio.findById(1L))
            .thenReturn(Optional.of(ana));

        servicio.subirSalario(1L, 10);

        assertThat(ana.getSalario())
            .isEqualByComparingTo("3300");
        verify(repositorio).save(ana);
    }
}
```

- @Mock crea un doble del repositorio.
- @InjectMocks crea el servicio y le inyecta los mocks.
- when(…).thenReturn(…) fija la respuesta del mock.
- assertThat (AssertJ) comprueba el resultado.
- verify comprueba que se llamó a save.
- Estructura Arrange-Act-Assert (Given-When-Then).

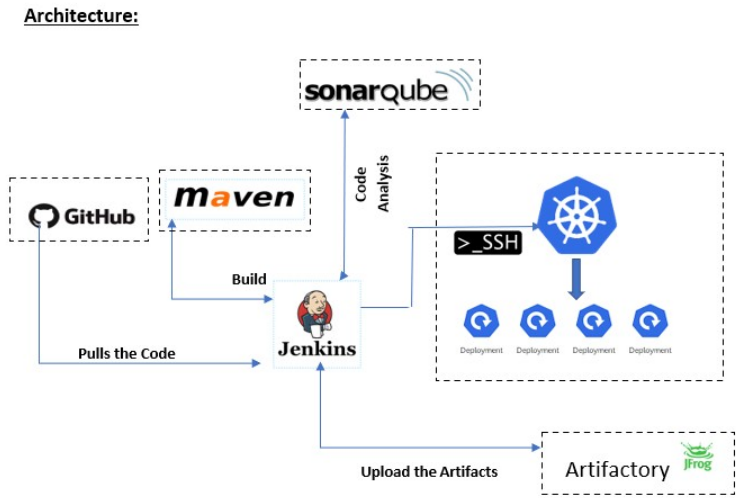
TIPOS DE PRUEBAS Y HERRAMIENTAS

Tipo	Objetivo	Herramientas
Unitarias	Una clase aislada, con dependencias simuladas	JUnit, Mockito, AssertJ
Integración	Varias capas juntas o con una BD real	Spring Boot Test, Testcontainers
Contrato	Que productor y consumidor de una API encajen	Pact, Spring Cloud Contract
API	Probar endpoints REST de extremo a extremo	REST Assured, Postman
Interfaz (E2E)	Flujos completos desde el navegador	Selenium, Playwright
Rendimiento	Tiempo de respuesta y carga soportada	JMeter, Gatling
Cobertura	Qué código ejecutan las pruebas	JaCoCo
Mutación	Calidad de las propias pruebas	PIT (Pitest)
BDD	Escenarios en lenguaje natural (Given/When/Then)	Cucumber

- Pirámide de pruebas: muchas unitarias, menos de integración y pocas de extremo a extremo.

INTEGRACIÓN CONTINUA (CI)

- Jenkins: predominante en la AGE, pipelines declarativos/scripted.
- Alternativas SaaS: GitHub Actions, GitLab CI, Azure DevOps.
- Funciones típicas:
  - Compilar (Maven/Gradle).
  - Ejecutar tests y recolectar resultados.
  - Analizar calidad (SonarQube).
  - Desplegar artefactos.



CONTINUOUS DELIVERY (CD)

- Herramientas:
  - Maven Release Plugin.
  - Ansible / scripts de despliegue.
  - Helm para empaquetar despliegues en Kubernetes.
  - GitOps con Argo CD o Flux: el repositorio Git es la fuente de verdad del despliegue. Tekton para pipelines en Kubernetes.
- Pipelines Docker: docker build + docker push (o Jib/buildpacks sin Dockerfile).
- Estrategias de despliegue: rolling update, blue-green y canary.
- Entrega continua (paso a producción con aprobación manual) frente a despliegue continuo (automático).

PIPELINE CI/CD TÍPICO PARA JAVA

Etapa	Qué se hace	Herramientas
1. Commit	Un push o una merge request lanza el pipeline	Git (GitLab, GitHub)
2. Compilación	Compilar y empaquetar	Maven, Gradle
3. Pruebas	Unitarias e integración; cobertura	JUnit, Testcontainers, JaCoCo
4. Calidad y seguridad	Análisis estático, quality gate, CVE en dependencias, SBOM	SonarQube, OWASP Dependency-Check, CycloneDX
5. Artefacto	Publicar el JAR y la imagen de contenedor	Nexus o Artifactory, Jib, buildpacks
6. Preproducción	Desplegar y lanzar pruebas de API, E2E y rendimiento	Helm, Argo CD, REST Assured, JMeter
7. Producción	Despliegue controlado y monitorización	Blue-green o canary; Prometheus, Grafana

CALIDAD DE CÓDIGO

- SonarQube (Server o Cloud, antes SonarCloud) para análisis estático: bugs, vulnerabilidades, code smells, duplicaciones.
- Quality Gates: el build falla si no se cumplen los umbrales (el perfil Sonar way exige 80% de cobertura en el código nuevo) o hay vulnerabilidades críticas.
- SonarQube for IDE (antes SonarLint) detecta los problemas mientras se escribe.
- Plugins adicionales:
  - SpotBugs, PMD, Checkstyle.
  - Integración con Maven/Gradle.
- AGE: exige informes Sonar en licitaciones TIC.

HERRAMIENTAS DE ANÁLISIS DE CÓDIGO

Herramienta	Analiza	Detecta
Checkstyle	Código fuente	Incumplimientos de estilo y convenciones (nombres, formato, Javadoc)
PMD	Código fuente	Malas prácticas, código muerto, complejidad; duplicados con CPD
SpotBugs	Bytecode	Errores probables (nulos, recursos sin cerrar); seguridad con FindSecBugs
Error Prone	Compilación	Errores comunes detectados por el propio compilador (Google)
SonarQube	Código fuente (multilenguaje)	Bugs, vulnerabilidades, code smells, puntos críticos de seguridad, deuda técnica
JaCoCo	Ejecución de pruebas	Cobertura de líneas y ramas
OWASP Dependency-Check	Dependencias	Librerías con vulnerabilidades conocidas (CVE)

CONTROL DE VERSIONES Y CONTENEDORES

- Git: estándar (GitHub, GitLab, forjas AGE).
  - Branching, merges, pull requests.
- Contenedores en desarrollo:
  - Docker Compose para levantar app + DB local.
  - Testcontainers para entornos de prueba.
  - Skaffold/Tilt para desarrollo iterativo en K8s.

GIT: ESTRATEGIAS DE RAMAS

Estrategia	Ramas	Cuándo encaja
GitFlow	main, develop, feature/*, release/*, hotfix/*	Versiones planificadas y entregas formales
GitHub Flow	main + ramas cortas de funcionalidad con pull request	Despliegue continuo, equipos pequeños
GitLab Flow	main + ramas por entorno (preproducción, producción)	Promoción controlada entre entornos
Trunk-based	Integración diaria en main, ramas muy cortas y feature flags	CI/CD maduro con buena batería de pruebas

- Revisión de código mediante pull o merge requests antes de integrar.
- Mensajes de commit con convención (Conventional Commits) y etiquetas para cada versión (v1.4.0).
- Forjas habituales: GitLab, GitHub y Azure DevOps.

4. ACCESO A DATOS E INTEGRACIÓN

JDBC · JPA y Spring Data · Transacciones · NoSQL · Servicios web y mensajería

ACCESO A DATOS Y APIS DE INTEGRACIÓN EN JAVA

- Persistencia de datos y comunicación con otros sistemas = pilares en aplicaciones empresariales.
- Java ofrece APIs estándar y frameworks para:
  - Bases de datos relacionales (JDBC, JPA/Hibernate).
  - NoSQL (Mongo, Cassandra, Redis con drivers Java/Spring Data).
  - Mensajería (JMS, Kafka, RabbitMQ).
  - Servicios web (JAX-WS/SOAP, JAX-RS/REST).
  - Integración moderna: GraphQL, WebSockets, gRPC.

JDBC

- Presente desde Java 1.1 (1997). Última versión: JDBC 4.3.
- Proporciona:
  - Conexiones (DriverManager, DataSource).
  - Sentencias SQL (Statement, PreparedStatement, CallableStatement).
  - ResultSet para leer resultados.
  - Control de transacciones (commit/rollback).
  - Metadatos de BD.
- Drivers JDBC modernos (PostgreSQL, Oracle, MySQL, etc.) se cargan automáticamente.

JDBC: TIPOS DE DRIVER Y PASOS

Tipo	Descripción	Uso actual
Tipo 1	Puente JDBC-ODBC	Eliminado en Java 8
Tipo 2	API nativa: usa librerías cliente del SGBD	Residual
Tipo 3	Protocolo de red a través de un servidor intermedio	Residual
Tipo 4	Java puro: habla directamente el protocolo de la BD	El estándar actual

1. Obtener la conexión de un DataSource con pool (HikariCP, el pool por defecto en Spring Boot).
2. Preparar la sentencia con parámetros (PreparedStatement).
3. Ejecutar: executeQuery (consultas) o executeUpdate (modificaciones).
4. Recorrer el ResultSet.
5. Cerrar los recursos con try-with-resources.
6. Controlar la transacción: setAutoCommit(false), commit() y rollback().

JPA/ORM: MAPEO OBJETO-RELACIONAL

- JPA (Java Persistence API, hoy Jakarta Persistence): estándar desde 2006 (Java EE 5); versión 3.2 en Jakarta EE 11.
- Modelo declarativo: entidades anotadas (@Entity, @Table, @Column).
- Relaciones (@OneToMany, @ManyToOne).
- CRUD con EntityManager; consultas con JPQL o Criteria API.
- Implementaciones:
  - Hibernate ORM es dominante (versión 7, con licencia Apache 2.0).
  - EclipseLink fue la implementación de referencia.
  - OpenJPA, DataNucleus son residuales.

CICLO DE VIDA DE UNA ENTIDAD JPA

Estado	Descripción	Cómo se llega
Nuevo (transient)	Objeto creado con new, sin relación con la BD	new Empleado(…)
Gestionado (managed)	Asociado al contexto de persistencia: sus cambios se sincronizan al hacer flush o commit	persist(), find(), consultas; merge() devuelve una copia gestionada
Separado (detached)	Tiene identidad en la BD, pero ya no está en el contexto	Cierre del EntityManager, detach(), clear()
Eliminado (removed)	Marcado para borrarse en el siguiente flush	remove()

- **Contexto de persistencia:** caché de primer nivel; garantiza una única instancia por identidad en cada EntityManager.
- **Dirty checking:** no hace falta un "update": los cambios en entidades gestionadas se escriben al confirmar la transacción.

CARGA PEREZOSA Y PROBLEMA N+1

- FetchType: EAGER (carga inmediata) o LAZY (bajo demanda).
  - Por defecto: @ManyToOne y @OneToOne son EAGER; @OneToMany y @ManyToMany, LAZY.
- Problema N+1: una consulta para la lista y otra por cada elemento al recorrer una relación perezosa.
- Soluciones: JOIN FETCH, @EntityGraph, proyecciones a DTO o carga por lotes (@BatchSize).
- LazyInitializationException: acceder a una relación LAZY fuera de la transacción.
- Spring Boot activa por defecto Open Session in View (spring.jpa.open-in-view); conviene desactivarlo y revisar el SQL generado.

```
// 1 consulta para los clientes + N para sus pedidos
for (Cliente c : clientes) c.getPedidos().size();

// Una sola consulta con JOIN FETCH
select c from Cliente c join fetch c.pedidos
```

JDBC VS JPA/HIBERNATE

- Con JDBC:

```
Connection conn = dataSource.getConnection();
PreparedStatement ps = conn.prepareStatement(
    "INSERT INTO empleado (nombre, salario) VALUES (?, ?)"
);
ps.setString(1, "Ana");
ps.setDouble(2, 3000.0);
ps.executeUpdate();
```

JDBC VS JPA/HIBERNATE(II)

- Con JPA/Hibernate:

```
Empleado emp = new Empleado("Ana", 3000.0);
entityManager.persist(emp);
```

SPRING DATA JPA Y JAKARTA DATA

```
public interface EmpleadoRepository
        extends JpaRepository<Empleado, Long> {

    // Consulta derivada del nombre del método
    List<Empleado> findByDepartamentoNombre(String nombre);

    // Consulta JPQL explícita con paginación
    @Query("select e from Empleado e where e.salario > ?1")
    Page<Empleado> conSalarioMayor(BigDecimal minimo,
                                   Pageable pagina);
}
```

- Spring Data genera la implementación en tiempo de ejecución.
- JpaRepository aporta save, findById, findAll, delete…
- Consultas derivadas: findBy…, countBy…, existsBy…
- Paginación y orden con Pageable y Sort.
- Jakarta Data 1.0 (Jakarta EE 11) estandariza este modelo: @Repository, @Find, @Query, @Save.

TRANSACCIONES: PROPAGACIÓN

Propagación	Comportamiento
REQUIRED (por defecto)	Usa la transacción existente o crea una nueva
REQUIRES_NEW	Suspende la actual y crea siempre una nueva
SUPPORTS	Usa la existente si la hay; si no, se ejecuta sin transacción
MANDATORY	Exige una transacción existente; si no la hay, lanza excepción
NOT_SUPPORTED	Se ejecuta sin transacción (suspende la actual)
NEVER	Lanza excepción si existe una transacción
NESTED	Transacción anidada con punto de guardado (savepoint)

- Propiedades ACID: atomicidad, consistencia, aislamiento y durabilidad.
- Con @Transactional, Spring deshace la transacción ante excepciones no comprobadas (RuntimeException), no ante las comprobadas salvo que se indique rollbackFor.
- Transacciones distribuidas: JTA/XA; en microservicios se prefieren las sagas.

TRANSACCIONES: NIVELES DE AISLAMIENTO

Nivel	Lectura sucia	Lectura no repetible	Lectura fantasma
READ UNCOMMITTED	Posible	Posible	Posible
READ COMMITTED	No	Posible	Posible
REPEATABLE READ	No	No	Posible
SERIALIZABLE	No	No	No

- **Lectura sucia:** leer datos que otra transacción aún no ha confirmado. **No repetible:** releer una fila y obtener otro valor. **Fantasma:** repetir una consulta y que aparezcan o desaparezcan filas.
- Por defecto: READ COMMITTED en Oracle, PostgreSQL y SQL Server; REPEATABLE READ en MySQL (InnoDB).
- En JPA: bloqueo optimista con @Version y pesimista con LockModeType.PESSIMISTIC_WRITE.

OTRAS FORMAS DE ACCESO A DATOS

- Bases NoSQL (drivers Java + Spring Data)
  - MongoDB: driver nativo + Spring Data MongoDB.
  - Redis: Lettuce (por defecto en Spring Data Redis), Jedis o Redisson; caché y sesiones.
  - Cassandra: driver Java de Apache Cassandra, Spring Data Cassandra.
  - Neo4j: driver + Spring Data Neo4j.
  - Elasticsearch: Elasticsearch Java API Client y Spring Data Elasticsearch (Jest está abandonado).
- Plantillas y librerías
  - JdbcTemplate / NamedParameterJdbcTemplate y JdbcClient (Spring): simplifican JDBC, más ligero que JPA.
  - jOOQ: SQL type-safe en Java, útil en entornos que requieren queries complejas.
  - MyBatis: mapea SQL escrito a mano a objetos.
- Migraciones del esquema: Flyway y Liquibase versionan los cambios de la BD.

APIS DE INTEGRACIÓN EN JAVA (I)

- Servicios web
  - SOAP (JAX-WS, hoy Jakarta XML Web Services): en retroceso, pero vigente en banca y AGE (ej. @firma).
  - REST (Jakarta REST, antes JAX-RS): estándar actual en Jakarta EE.
  - Spring REST Controllers son la alternativa más usada en proyectos Boot.
  - Clientes HTTP: RestClient y WebClient (Spring), HttpClient del JDK.
- Mensajería y tiempo real
  - JMS (hoy Jakarta Messaging): colas/tópicos (ActiveMQ, IBM MQ).
  - Kafka, RabbitMQ: APIs y soporte Spring.
  - WebSockets (Jakarta WebSocket, antes JSR 356): comunicación bidireccional en tiempo real.

APIS DE INTEGRACIÓN EN JAVA (II)

- JSON
  - Jakarta JSON Processing (JSON-P) y JSON Binding (JSON-B).
  - Jackson (por defecto en Spring; Jackson 3 en Spring Boot 4) y Gson.
- XML (aún muy presente en e-Government)
  - JAXB (Jakarta XML Binding): mapeo objeto-XML; desde Java 11 ya no viene en el JDK.
  - StAX, DOM y SAX (JAXP, incluidos en el JDK).
  - Firma electrónica XML (XAdES) y WS-Security en integraciones con la Administración.
- Otros formatos
  - Apache POI (Excel, Word), OpenCSV y Apache Commons CSV.
  - Apache PDFBox y OpenPDF para PDF.
  - Protocol Buffers y Avro: formatos binarios para gRPC y Kafka.

MENSAJERÍA: JMS, AMQP Y KAFKA

Aspecto	JMS / AMQP (ActiveMQ, RabbitMQ, IBM MQ)	Apache Kafka
Modelo	Colas (punto a punto) y tópicos (publicación/suscripción)	Registro de eventos distribuido, con topics divididos en particiones
Consumo	El mensaje se elimina al confirmarse	Los mensajes se conservan un tiempo; cada grupo de consumidores guarda su posición (offset)
Orden	Por cola	Garantizado dentro de cada partición
Escalado	Varios consumidores compiten por la cola	Paralelismo por particiones dentro de un grupo
Uso típico	Integración entre aplicaciones, tareas asíncronas	Streaming de eventos, alto volumen, event sourcing
En Java	Jakarta Messaging, Spring JMS, Spring AMQP	Kafka Clients, Spring for Apache Kafka, Kafka Streams

- Garantías de entrega: at-most-once, at-least-once y exactly-once (Kafka, con productores idempotentes y transacciones).

REST VS SOAP EN JAVA(I)

Aspecto	REST (JAX-RS / Spring REST)	SOAP (JAX-WS)
Formato de datos	JSON / XML (ligero, flexible)	XML estructurado (WSDL obligatorio)
Protocolo	Usa HTTP directamente (GET, POST, PUT, DELETE)	Encapsula mensajes en sobres SOAP sobre HTTP, SMTP, etc.
Contratos	Opcional (OpenAPI/Swagger)	Obligatorio con WSDL (contrato rígido)
Facilidad de uso	Simple de implementar y probar (curl, Postman)	Más complejo, requiere tooling específico
Rendimiento	Generalmente más rápido (payloads pequeños)	Más pesado (XML verboso, headers extra)
Casos típicos	Microservicios modernos, APIs públicas, apps móviles	Banca, e-Gov, integraciones legacy

REST VS SOAP EN JAVA (II)

- REST (Jakarta REST / Spring RestController):
  - Simplicidad: usa HTTP directamente (GET, POST, etc.).
  - Intercambia JSON/XML ligero.
  - Fácil de probar con curl/Postman.
  - Idóneo para microservicios modernos.

```
@Path("/empleados")
public class EmpleadoResource {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<Empleado> listar() {
        return servicioEmpleado.obtenerTodos();
    }
}
```

- SOAP (JAX-WS con WSDL):
  - Basado en XML y contratos rígidos (WSDL).
  - Usa protocolos complejos (envelopes, headers).
  - Estandarizado (WS-Security, WS-Addressing).
  - Persistente en banca, e-gov y sistemas legados.

```
@WebService
public class EmpleadoService {

    @WebMethod
    public List<Empleado> listarEmpleados() {
        return servicioEmpleado.obtenerTodos();
    }
}
```

FRAMEWORKS DE INTEGRACIÓN EMPRESARIAL

- Apache Camel (versión 4): implementa los patrones de integración empresarial (EIP) con cientos de conectores.
- Spring Integration es el equivalente en ecosistema Spring.
- Patrones EIP habituales: enrutado según contenido, traductor de mensajes, divisor y agregador, cola de mensajes fallidos (dead letter).
- Jakarta Connectors (JCA) permite conectar sistemas legacy.
- Jakarta Mail para envío de notificaciones por email.
- Procesos por lotes: Spring Batch y Jakarta Batch (cargas masivas, ficheros nocturnos).

5. SEGURIDAD EN EL DESARROLLO JAVA

APIs del JDK · Autenticación · OAuth 2.0 y JWT · OWASP Top 10 · Cadena de suministro · Normativa

SEGURIDAD EN EL DESARROLLO JAVA

- Java nace con seguridad en mente:
  - Manejo automático de memoria (sin los desbordamientos de búfer típicos de C/C++).
  - Verificación del bytecode y tipado fuerte.
  - Sandboxing (applets): obsoleto. El Security Manager está desactivado desde Java 24 y la API de applets se eliminó en Java 26.
- A nivel de aplicaciones es responsabilidad del dev:
  - Autenticación y autorización.
  - Cifrado y protección de datos.
  - Prevención de vulnerabilidades comunes.

APIS DE SEGURIDAD EN EL JDK

- JCA (Java Cryptography Architecture):
  - Cifrado AES, RSA/ECC, SHA-256, firmas digitales.
  - KeyStore, Cipher, Signature, MessageDigest.
  - Criptografía poscuántica: ML-KEM y ML-DSA desde Java 24; API de derivación de claves (KDF) en Java 25.
- TLS (JSSE): HTTPS integrado en servidores/clientes Java; TLS 1.3 desde Java 11.
- Gestión de certificados: KeyStore para .p12 / .jks (ej. FNMT); PKCS#12 es el formato por defecto desde Java 9.
- Usos en AGE: firma digital (ej. AutoFirma).

AUTENTICACIÓN Y AUTORIZACIÓN

- Spring Security: login, OAuth2, JWT, control de acceso por URL y por método (@PreAuthorize).
- Jakarta Security / MicroProfile JWT: @RolesAllowed, SecurityContext.
- Single Sign-On (Cl@ve, SAML, OAuth2/OIDC): integraciones vía Keycloak o adaptadores.
- Spring Authorization Server permite montar un servidor OAuth2/OIDC propio; desde 2025 forma parte de Spring Security 7.
- Buenas prácticas: contraseñas con hash adaptativo y sal (Argon2, BCrypt, PBKDF2), roles bien definidos, doble factor.

OAUTH 2.0, OPENID CONNECT Y JWT

- **OAuth 2.0:** autorización delegada entre cliente, servidor de autorización y servidor de recursos.
  - Authorization Code + PKCE: aplicaciones web, SPA y móviles (el recomendado).
  - Client Credentials: comunicación entre servicios, sin usuario.
  - Implicit y Resource Owner Password: desaconsejados (desaparecen en OAuth 2.1).
- **OpenID Connect:** autenticación sobre OAuth 2.0; añade el ID Token con la identidad del usuario.
- **JWT (RFC 7519):** token firmado (cabecera.carga.firma) con claims como iss, sub, aud y exp; se valida sin guardar sesión.
  - Va firmado, no cifrado: sin datos sensibles y con caducidad corta.
- Implementaciones: Keycloak y Spring Authorization Server.

VULNERABILIDADES Y PREVENCIÓN

- SQL Injection: usar PreparedStatement/JPA con parámetros.
- XSS: escapar datos en vistas (Thymeleaf lo hace por defecto) y cabecera CSP.
- CSRF: tokens (Spring Security los activa por defecto).
- Deserialización insegura: no deserializar datos no confiables; filtros de deserialización del JDK.
- Log4Shell (2021, CVE-2021-44228) y Spring4Shell (2022, CVE-2022-22965): ejemplos críticos de dependencia vulnerable.
- Mitigación: mantener librerías actualizadas, OWASP Dependency-Check, SBOM.

OWASP TOP 10 (2025)

Código	Categoría	Ejemplo en Java y prevención
A01	Control de acceso roto	Endpoints sin comprobar permisos: @PreAuthorize, denegar por defecto
A02	Configuración de seguridad incorrecta	Actuator expuesto, trazas de error al usuario
A03	Fallos en la cadena de suministro de software	Dependencias vulnerables o manipuladas: SBOM, firmas
A04	Fallos criptográficos	Algoritmos débiles, datos sin cifrar: TLS 1.2+, AES-GCM
A05	Inyección	SQL, JPQL, LDAP o comandos: consultas parametrizadas
A06	Diseño inseguro	Sin modelado de amenazas ni requisitos de seguridad
A07	Fallos de autenticación	Contraseñas débiles, sesiones mal gestionadas: MFA
A08	Fallos de integridad de software o datos	Deserialización insegura, actualizaciones sin firmar
A09	Fallos de registro y alertas	Sin trazas de seguridad ni alertas
A10	Mal manejo de condiciones excepcionales	Errores que dejan el sistema inseguro o filtran información

Novedades frente a 2021: entran la cadena de suministro (A03) y las condiciones excepcionales (A10); SSRF se integra en A01.

SEGURIDAD DE LA CADENA DE SUMINISTRO

- Una aplicación Java típica incluye decenas o cientos de dependencias transitivas: la mayor parte del código no es propio.
- **SBOM** (lista de materiales del software) en formato CycloneDX o SPDX, generada en el build con plugins de Maven o Gradle.
- **Vulnerabilidades:** OWASP Dependency-Check; Dependabot o Renovate para actualizar; Trivy para imágenes de contenedor.
- **Integridad:** repositorios corporativos como proxy, verificación de checksums y firmas (PGP en Maven Central, Sigstore).
- **Contenedores:** imágenes base mínimas, sin usuario root y escaneadas antes de desplegar.
- **Normativa europea:** el Reglamento de Ciberresiliencia (CRA) obliga a notificar vulnerabilidades explotadas desde septiembre de 2026 y se aplica por completo en diciembre de 2027.

AUDITORÍA, NORMATIVAS Y CALIDAD

- ENS (Real Decreto 311/2022): cifrado robusto, trazabilidad, control de accesos.
- RGPD y LOPDGDD (LO 3/2018): protección de datos desde el diseño y por defecto, registro de accesos a datos personales.
- Logs y auditoría: no registrar datos sensibles; auditorías de negocio (AOP, interceptores).
- Herramientas:
  - SonarQube (reglas de seguridad).
  - Fortify, Checkmarx, Veracode (auditorías avanzadas).
  - Pruebas dinámicas (DAST) con ZAP.

6. CERTIFICACIONES

Oracle Java SE · Spring · Kubernetes y ecosistema

CERTIFICACIONES JAVA

- Certificaciones que acreditan conocimientos formales.
- Oracle es el proveedor principal (Java SE, EE).
- También existen certificaciones de frameworks (Spring) y tecnologías asociadas (K8s, Big Data).

ORACLE JAVA SE (STANDARD EDITION)

- OCP Java SE 21 Developer (1Z0-830): la más reciente.
  - Lenguaje base: sintaxis, APIs core, streams, concurrencia, módulos.
  - Examen único: sustituyó a las antiguas OCA + OCP.
  - Certificación más demandada, junto a la de Java SE 17 (1Z0-829).
- Otras vigentes: Java SE 11 (1Z0-819) y Java Foundations (1Z0-811), de nivel inicial.
- Plataforma empresarial:
  - Java EE 7 Application Developer (1Z0-900): sigue en el catálogo, aunque se basa en una versión antigua.
  - Las certificaciones Master/Architect de Java EE ya no figuran en el catálogo de Oracle.

CERTIFICACIONES DE FRAMEWORKS Y ECOSISTEMA

- Spring Certified Professional (examen 2V0-72.22, hoy gestionado por Broadcom)
  - Ecosistema Spring: Boot, Data, Security, REST.
  - Muy valorada en proyectos Spring Boot.
- CKAD / CKA / CKS (Linux Foundation)
  - Kubernetes para despliegues cloud-native es cada vez más relevante para dev Java.
- Big Data: Hadoop/Spark (Cloudera, que absorbió a Hortonworks en 2019; Databricks).
- Otras: Helidon Microservices Developer (Oracle, 1Z0-1113), menos extendidas.

RESUMEN DE CERTIFICACIONES

Certificación	Código	Contenido
Java Foundations	1Z0-811	Nivel inicial: fundamentos del lenguaje
Java SE 8 Programmer I y II	1Z0-808 / 1Z0-809	Antiguas OCA y OCP de Java 8
Java SE 11 Developer	1Z0-819	Profesional (Java 11)
Java SE 17 Developer	1Z0-829	Profesional (Java 17)
Java SE 21 Developer Professional	1Z0-830	Profesional (Java 21), la más reciente
Java EE 7 Application Developer	1Z0-900	Desarrollo empresarial (plataforma antigua)
Helidon Microservices Developer	1Z0-1113	Microservicios con Helidon
Spring Certified Professional	2V0-72.22	Spring Framework y Spring Boot
CKAD / CKA / CKS	—	Kubernetes: desarrollador, administrador y seguridad

7. JAVA EN LA ADMINISTRACIÓN PÚBLICA

Casos de uso · Soluciones comunes · Marco normativo

CASOS DE USO DE JAVA EN LA ADMINISTRACIÓN PÚBLICA

- Java está en el núcleo de los sistemas públicos digitales.
- Se usa tanto en aplicaciones de ciudadano como en sistemas internos.
- Razones: portabilidad, fiabilidad, estándares abiertos, comunidad amplia.

EJEMPLOS EMBLEMÁTICOS

- AutoFirma (Cliente @Firma)
  - App de escritorio Java (Swing).
  - Firma electrónica multi-plataforma (Windows/Linux/Mac).
  - Base de decenas de procedimientos telemáticos.
- Portafirmas
  - Flujo de firma electrónica en organismos.
  - Java EE + JSF/PrimeFaces.
  - Reutilizado en ministerios y entes públicos.

SISTEMAS CRÍTICOS NACIONALES

- AEAT (Agencia Tributaria)
  - Backoffice en Java EE (WebLogic).
  - Procesa millones de declaraciones.
- Plataforma de Intermediación de Datos (PID)
  - Bus de servicios para intercambiar datos entre AAPP.
  - SOAP/WS-Security en origen, ahora también REST.
- Notific@
  - Sistema centralizado de notificaciones electrónicas.
  - Basado en Java + mensajería (JMS, colas).

SOLUCIONES COMUNES REUTILIZABLES

Solución	Función
Cl@ve	Identificación y firma de la ciudadanía (certificado electrónico, Cl@ve PIN, Cl@ve permanente…)
@firma / AutoFirma	Validación de certificados y firmas; cliente de firma de escritorio
Port@firmas	Circuitos de firma electrónica de documentos en los organismos
Notific@ / DEHú	Notificaciones electrónicas y Dirección Electrónica Habilitada única
Plataforma de Intermediación de Datos	Consulta de datos entre Administraciones (protocolo SCSP) para no pedirlos al ciudadano
SIR / GEISER	Intercambio de asientos registrales y registro electrónico
InSide / Archive	Documento y expediente electrónico (ENI) y archivo definitivo

Disponibles en el Centro de Transferencia de Tecnología (CTT); las aplicaciones Java las integran mediante servicios web o librerías cliente.

MARCO NORMATIVO EN LA AGE

Norma	Relevancia para el desarrollo
Ley 39/2015	Procedimiento administrativo electrónico: relación electrónica, registro y notificaciones
Ley 40/2015	Reutilización de sistemas y aplicaciones (art. 157) y Centro de Transferencia de Tecnología (art. 158)
RD 203/2021	Reglamento de actuación y funcionamiento del sector público por medios electrónicos
RD 311/2022 (ENS)	Medidas de seguridad según la categoría del sistema (básica, media o alta)
RD 4/2010 (ENI)	Interoperabilidad: formatos, documento y expediente electrónico, normas técnicas
RGPD y LOPDGDD	Protección de datos desde el diseño y por defecto
RD 1112/2018	Accesibilidad de sitios web y aplicaciones móviles del sector público
Ley 9/2017 (LCSP)	Contratación pública: pliegos que fijan tecnologías, calidad y seguridad

BALANCE Y FUTURO

- Java = lenguaje transversal en la AGE:
  - Firma, notificación, registro, autenticación.
  - Gestión económica, RRHH, expedientes.
- Open source y reutilización (Ley 40/2015: CTT).
- Retos actuales: modernización de monolitos JEE hacia contenedores/microservicios.
- Futuro: seguirá siendo pilar en transformación digital, big data (Spark), IA (Spring AI, LangChain4j) y cloud-native.

REPASO

Ideas clave · Preguntas tipo test

IDEAS CLAVE (I)

- De Servlets/JSP a frameworks MVC: Struts (en desuso), JSF (hoy Jakarta Faces) y Spring MVC.
- Spring Boot, estándar de facto: autoconfiguración, starters y servidor embebido; hoy rama 4.x (Java 17+).
- Jakarta EE 9 cambió javax.* por jakarta.*; Jakarta EE 11 (2025) exige Java 17 y añade Jakarta Data.
- Tendencia: SPA + backend Java con APIs REST; también GraphQL, gRPC, WebSockets y SSE.
- Microservicios autónomos con sus propios datos: API Gateway, Circuit Breaker, Saga.
- Kubernetes asume descubrimiento y escalado; Quarkus, Micronaut y Helidon reducen arranque y memoria.

IDEAS CLAVE (II)

- JDK: LTS cada dos años (8, 11, 17, 21, 25); hilos virtuales desde Java 21.
- IntelliJ IDEA lidera; Maven es el estándar en la AGE y Gradle crece; JUnit, Mockito y Testcontainers.
- CI/CD: Jenkins o GitLab CI, SonarQube con quality gates, Helm o Argo CD para desplegar.
- Datos: JDBC y JPA/Hibernate; cuidado con el N+1 y la propagación de @Transactional.
- Seguridad: Spring Security, OAuth 2.0/OIDC con JWT, OWASP Top 10 y SBOM; marco ENS y RGPD.
- En la AGE, Java sustenta servicios comunes como @firma, Cl@ve o Notific@.

PREGUNTA TIPO TEST 1/12

¿Qué cambio introdujo Jakarta EE 9?

a) Exigir Java 17 como versión mínima

b) Cambiar el espacio de nombres de javax.* a jakarta.* ✓

c) Incorporar la especificación Jakarta Data

d) Eliminar JSF de la plataforma

Justificación: Jakarta EE 9 (2020) solo cambió el espacio de nombres; Java 17 y Jakarta Data llegaron con Jakarta EE 11.

PREGUNTA TIPO TEST 2/12

En Spring MVC, ¿qué componente actúa como Front Controller?

a) ViewResolver

b) HandlerMapping

c) DispatcherServlet ✓

d) ActionServlet

Justificación: ActionServlet era el Front Controller de Struts 1.

PREGUNTA TIPO TEST 3/12

¿Qué método HTTP es idempotente pero no seguro?

a) GET

b) POST

c) PUT ✓

d) HEAD

Justificación: PUT modifica el estado del servidor, pero repetirlo produce el mismo resultado.

PREGUNTA TIPO TEST 4/12

¿Cuál es la propagación por defecto de @Transactional en Spring?

a) REQUIRES_NEW

b) REQUIRED ✓

c) SUPPORTS

d) MANDATORY

Justificación: REQUIRED usa la transacción existente o crea una nueva si no la hay.

PREGUNTA TIPO TEST 5/12

¿Qué tipo de carga usa JPA por defecto en una relación @ManyToOne?

a) LAZY

b) EAGER ✓

c) Depende del proveedor

d) BATCH

Justificación: @ManyToOne y @OneToOne son EAGER por defecto; @OneToMany y @ManyToMany, LAZY.

PREGUNTA TIPO TEST 6/12

¿Qué nivel de aislamiento evita las lecturas sucias, pero permite lecturas no repetibles?

a) READ UNCOMMITTED

b) READ COMMITTED ✓

c) REPEATABLE READ

d) SERIALIZABLE

Justificación: REPEATABLE READ evita también las lecturas no repetibles; solo SERIALIZABLE evita las fantasma.

PREGUNTA TIPO TEST 7/12

En Maven, ¿qué ámbito corresponde a la API de Servlet que aporta el servidor de aplicaciones?

a) compile

b) runtime

c) provided ✓

d) test

Justificación: provided: se usa para compilar, pero no se empaqueta porque la proporciona el servidor.

PREGUNTA TIPO TEST 8/12

¿Qué anotación de JUnit 5/6 sustituye a @Before de JUnit 4?

a) @BeforeAll

b) @BeforeEach ✓

c) @Setup

d) @Init

Justificación: @BeforeAll sustituye a @BeforeClass y se ejecuta una sola vez por clase.

PREGUNTA TIPO TEST 9/12

¿En qué versión de Java se incorporaron de forma definitiva los hilos virtuales?

a) Java 17

b) Java 19

c) Java 21 ✓

d) Java 25

Justificación: En Java 19 y 20 estaban en preview; son definitivos desde Java 21 (JEP 444).

PREGUNTA TIPO TEST 10/12

¿Qué flujo de OAuth 2.0 es el adecuado para la comunicación entre dos servicios sin usuario?

a) Authorization Code + PKCE

b) Implicit

c) Client Credentials ✓

d) Resource Owner Password

Justificación: Authorization Code + PKCE es para aplicaciones con usuario; Implicit y Password están desaconsejados.

PREGUNTA TIPO TEST 11/12

¿Qué categoría encabeza el OWASP Top 10 de 2025?

a) Inyección

b) Control de acceso roto ✓

c) Fallos criptográficos

d) Fallos en la cadena de suministro de software

Justificación: Control de acceso roto repite en primer lugar; la cadena de suministro entra en el tercero.

PREGUNTA TIPO TEST 12/12

¿Qué herramienta analiza el bytecode para detectar errores probables?

a) Checkstyle

b) PMD

c) SpotBugs ✓

d) JaCoCo

Justificación: Checkstyle y PMD analizan el código fuente; JaCoCo mide la cobertura de las pruebas.

¿PREGUNTAS?